

SQL Project : Expense Tracker

Built a personal expense tracker from scratch using SQLite. Created a table to log 15 days of expenses across 4 categories: Food, Transport, Entertainment, and Shopping.

Wrote queries to analyze the data:

- Total amount spent overall
- Total spending broken down by category using GROUP BY
- Most expensive single purchase using MAX()
- Filtered expenses over \$10 using WHERE
- Sorted all expenses from highest to lowest using ORDER BY
- Labeled each expense as Cheap, Moderate, or Expensive using CASE

Tools: SQLite, DB Browser for SQLite

1. Creating a table of expenses:

```
CREATE TABLE expenses (  
  id INTEGER PRIMARY KEY,  
  date TEXT,  
  category TEXT,  
  amount REAL,  
  description TEXT  
);
```

2. Adding values into the table of expenses:

```
INSERT INTO expenses VALUES (1, '2026-07-01', 'Food', 12.50, 'Chipotle');  
INSERT INTO expenses VALUES (2, '2026-07-02', 'Transport', 3.00, 'Bus fare');  
INSERT INTO expenses VALUES (3, '2026-07-03', 'Food', 8.75, 'Groceries');  
INSERT INTO expenses VALUES (4, '2026-07-04', 'Entertainment', 15.00, 'Movie ticket');  
INSERT INTO expenses VALUES (5, '2026-07-05', 'Shopping', 45.00, 'Shirt from H&M');
```

```

INSERT INTO expenses VALUES (6, '2026-07-06', 'Food', 6.50, 'Ramen');
INSERT INTO expenses VALUES (7, '2026-07-07', 'Transport', 3.00, 'Bus fare');
INSERT INTO expenses VALUES (8, '2026-07-08', 'Food', 22.00, 'Dinner with friends');
INSERT INTO expenses VALUES (9, '2026-07-09', 'Entertainment', 9.99, 'Spotify');
INSERT INTO expenses VALUES (10, '2026-07-10', 'Shopping', 18.00, 'Groceries at Target');
INSERT INTO expenses VALUES (11, '2026-07-11', 'Food', 5.25, 'Coffee and bagel');
INSERT INTO expenses VALUES (12, '2026-07-12', 'Transport', 25.00, 'Uber');
INSERT INTO expenses VALUES (13, '2026-07-13', 'Entertainment', 12.00, 'Netflix');
INSERT INTO expenses VALUES (14, '2026-07-14', 'Food', 30.00, 'Sushi');
INSERT INTO expenses VALUES (15, '2026-07-15', 'Shopping', 60.00, 'Shoes');

```

3. Checking to see if the values are inserted into the table:

```
SELECT * FROM expenses;
```

	id	date	category	amount	description
1	1	2026-07-01	Food	12.5	Chipotle
2	2	2026-07-02	Transport	3.0	Bus fare
3	3	2026-07-03	Food	8.75	Groceries
4	4	2026-07-04	Entertainment	15.0	Movie ticket
5	5	2026-07-05	Shopping	45.0	Shirt from H&M
6	6	2026-07-06	Food	6.5	Ramen
7	7	2026-07-07	Transport	3.0	Bus fare
8	8	2026-07-08	Food	22.0	Dinner with friends
9	9	2026-07-09	Entertainment	9.99	Spotify
10	10	2026-07-10	Shopping	18.0	Groceries at Target
11	11	2026-07-11	Food	5.25	Coffee and bagel
12	12	2026-07-12	Transport	25.0	Uber
13	13	2026-07-13	Entertainment	12.0	Netflix
14	14	2026-07-14	Food	30.0	Sushi
15	15	2026-07-15	Shopping	60.0	Shoes

4. Query to get the total amount spent overall:

```
SELECT SUM(amount) FROM expenses;
```

Answer: 275.99

5. Query to calculate the amount spent per category:

```
SELECT category, SUM(amount)
```

```
FROM expenses
```

```
GROUP BY category;
```

	category	SUM(amount)
1	Entertainment	36.99
2	Food	85.0
3	Shopping	123.0
4	Transport	31.0

6. Query to calculate the most expensive single purchase:

```
SELECT description, MAX(amount)
```

```
FROM expenses;
```

	description	MAX(amount)
1	Shoes	60.0

7. Query to get all expenses over \$10:

```
SELECT *  
FROM expenses  
WHERE amount > 10;
```

	id	date	category	amount	description
1	1	2026-07-01	Food	12.5	Chipotle
2	4	2026-07-04	Entertainment	15.0	Movie ticket
3	5	2026-07-05	Shopping	45.0	Shirt from H&M
4	8	2026-07-08	Food	22.0	Dinner with friends
5	10	2026-07-10	Shopping	18.0	Groceries at Target
6	12	2026-07-12	Transport	25.0	Uber
7	13	2026-07-13	Entertainment	12.0	Netflix
8	14	2026-07-14	Food	30.0	Sushi
9	15	2026-07-15	Shopping	60.0	Shoes

8. Query to get all expenses sorted by amount (highest to lowest):

```
SELECT *  
FROM expenses  
ORDER BY amount DESC;
```

	id	date	category	amount	description
1	15	2026-07-15	Shopping	60.0	Shoes
2	5	2026-07-05	Shopping	45.0	Shirt from H&M
3	14	2026-07-14	Food	30.0	Sushi
4	12	2026-07-12	Transport	25.0	Uber
5	8	2026-07-08	Food	22.0	Dinner with friends
6	10	2026-07-10	Shopping	18.0	Groceries at Target
7	4	2026-07-04	Entertainment	15.0	Movie ticket
8	1	2026-07-01	Food	12.5	Chipotle
9	13	2026-07-13	Entertainment	12.0	Netflix
10	9	2026-07-09	Entertainment	9.99	Spotify
11	3	2026-07-03	Food	8.75	Groceries
12	6	2026-07-06	Food	6.5	Ramen
13	11	2026-07-11	Food	5.25	Coffee and bagel
14	2	2026-07-02	Transport	3.0	Bus fare
15	7	2026-07-07	Transport	3.0	Bus fare

9. Query to label amount as 'cheap', 'moderate', and 'expensive':

```
SELECT amount,  
CASE  
WHEN amount < 10 THEN 'Cheap'  
WHEN amount >= 10 AND amount <= 30 THEN 'Moderate'  
WHEN amount > 30 THEN 'Expensive'  
END  
FROM expenses;
```

	amount	CASE WHEN amount < 10 THEN 'Cheap' WHEN amount >= 10 AND amount <= 30 THEN 'Moderate' WHEN amount > 30 THEN 'Expensive' END
1	12.5	Moderate
2	3.0	Cheap
3	8.75	Cheap
4	15.0	Moderate
5	45.0	Expensive
6	6.5	Cheap
7	3.0	Cheap
8	22.0	Moderate
9	9.99	Cheap
10	18.0	Moderate
11	5.25	Cheap
12	25.0	Moderate
13	12.0	Moderate
14	30.0	Moderate
15	60.0	Expensive